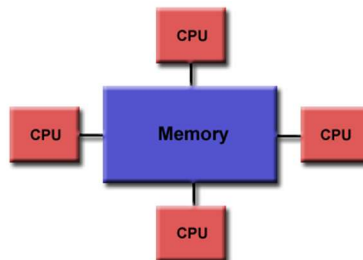


Тема вежбе: Паралелно програмирање

ПАРАЛЕЛНО ПРОГРАМИРАЊЕ – ОПЕН МП

Дељена меморија - концепт

Савремени вишепроцесорски рачунарски системи садрже више процесора који деле заједничку меморију. Како би се омогућило истовремено извршавање више делова програма, неопходно је писати програме који подржавају паралелно извршавање. У овој вежби бавићемо се паралелним програмирањем коришћењем OpenMP библиотеке, која омогућава једноставно увођење паралелизма у програме написане у језицима C/C++. Основни модел који користи OpenMP је модел дељене меморије, у којем више нити приступа истим подацима.



У моделу дељене меморије све нити имају приступ истом адресном простору. То значи да више нити може истовремено читати и писати исте променљиве. Ово омогућава једноставну комуникацију између нити, али уводи и ризик од грешака, као што је некоректно ажурирање података.

Петље су често најпогоднија места за увођење паралелизма у програмима. Уколико итерације у петљи немају међусобну зависност података, могуће их је расподелити различитим нитима и такве итерације се могу извршавати у било ком редоследу. Могуће је паралелизовати само оне петље код којих се број итерација може унапред израчунати и овакве петље називамо **добро формиране петље**.

Уколико имамо петљу као у следећем примеру и две нити, могуће је итерирати део од 0 до 49 једној, а део 50-99 другој нити:

```
for (i = 0; i < 100; i++)
{
    a[i] += b[i];
}
```

Свака итерација петље користи различите елементе низа, што значи да нема зависности између итерација. Због тога се оне могу извршавати паралелно без утицаја на исправност резултата.

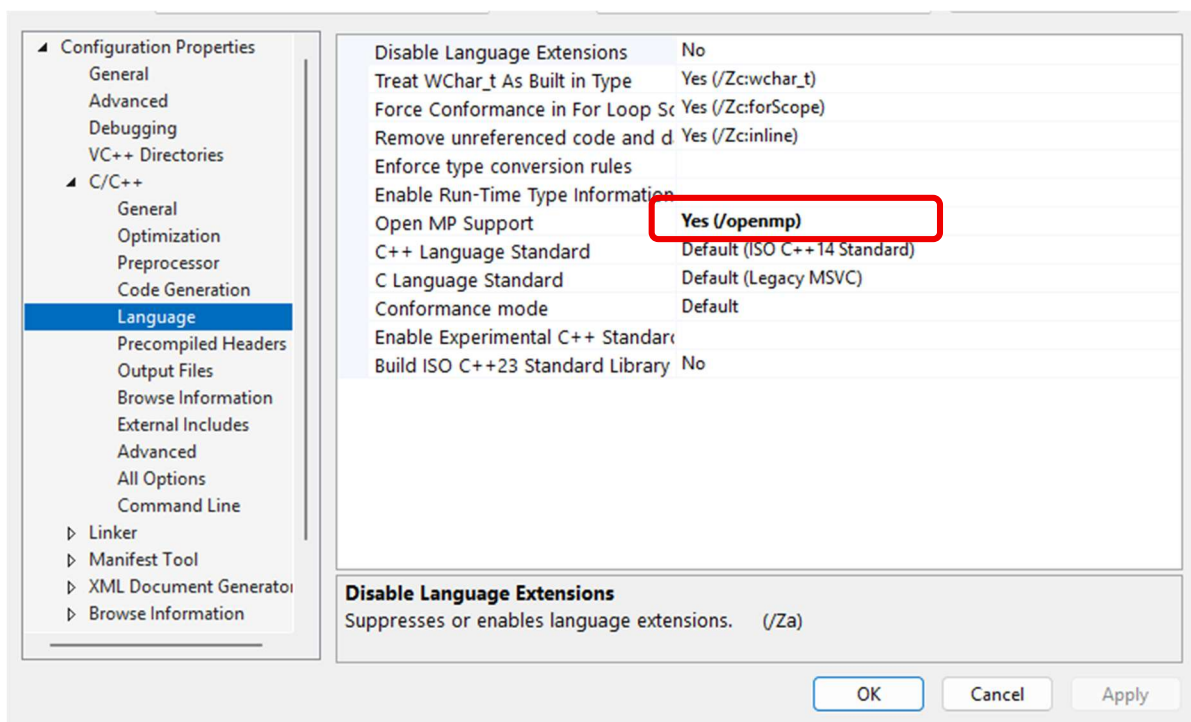
OpenMP

OpenMP (енгл. *Open Multi-Processing*) је једноставна програмска спрега – API (енгл. *Application programming interface*) која обухвата скуп директива и библиотечких функција које омогућавају да се постојећи секвенцијални програм релативно једноставно прошири паралелизмом. **OpenMP** није програмски језик, већ скуп надоградњи за програмске језике *Фортран*, *Ц* и *Ц++*. Већина савремених компајлера поседује уграђену подршку за OpenMP, што значајно олакшава његово коришћење.

Да би се успешно превео програмски код који користи OpenMP, неопходно је компајлеру проследити одговарајуће опције приликом превођења:

GCC: -fopenmp

У окружењу Visual Studio Community је потребно подесити OpenMP опцију на путањи: **Properties** → **C/C++** → **Language** → **OpenMP Support**



❗ **НАПОМЕНА:** Уколико се не дода одговарајућа опција приликом превођења, OpenMP директиве ће бити игнорисане – као да не постоје у коду!

Основне компоненте OpenMP надоградње су:

- **Компајлерске директиве** (енгл. *Compiler directives*)
- **Библиотеке са помоћним функцијама** (енгл. *Runtime library routines*)
- **Променљиве окружења** (енгл. *Environment variables*)

Компајлерске директиве:

Компајлерске директиве представљају посебне линије изворног кода, које служе за укључивање или искључивање одређених подешавања компајлера, и имају значење само преводиоцима који их подржавају (нису дефинисане стандардом језика). Преводиоци који их не подржавају, једноставно их **игноришу**. Директиве не мењају логику програма, већ само начин извршавања. Ако се уклоне, програм ће и даље радити, али без паралелизма.

Компајлерске директиве су увек у формату **#pragma <naziv direktive>**, док OpenMP директиве започињу са ознаком **omp**:

```
#pragma omp
```

Свака OpenMP директива може важити за:

- **Један исказ** (следећи ред у коду)
- **Блок кода** обухваћен витичастим заградама {}

Пример 1 – један исказ:

```
#pragma omp parallel
jedan iskaz (do sledeceg `;')
```

Пример 2 – блок кода обухваћен {}, који следи одмах након директиве.

```
#pragma omp parallel
{
    blok koda
}
```

 **Честа грешка:** Уколико заборавите заграде {} око више наредби, само прва наредба ће се извршити паралелно!

Помоћне функције из OpenMP библиотеке:

Помоћне функције и променљиве окружења служе за контролу и праћење паралелног извршавања. Све помоћне функције и променљиве окружења декларисане су у заглављу *omp.h* и могу се поделити у три групе:

1. Функције за добијање информација о нитима
2. Функције за контролу извршавања
3. Функције за мерење перформанси

Најчешће коришћене функције које нуди OpenMP су дате у Табели 1.1.

Функција	Опис
<code>int omp_get_num_threads();</code>	Добавља број тренутно коришћених нити. Враћа 1 ако се зове изван паралелног региона.
<code>int omp_get_thread_num();</code>	Добавља редни број нити која се тренутно извршава. Узима вредности од 0 до <code>omp_get_num_threads() - 1</code> .
<code>void omp_set_num_threads(int n);</code>	Подешава број нити које ће се користити
<code>int omp_in_parallel();</code>	Враћа 1 уколико је позвана унутар паралелног региона, 0 уколико је ван њега.
<code>int omp_get_max_threads();</code>	Враћа колико је укупно нити доступно за коришћење (максималан број нити за било који паралелни регион)
<code>double omp_get_wtime();</code>	Враћа време у секундама, протекло од одређеног тренутка у прошлости, али пошто време није апсолутно, потребно је израчунати разлику између <i>startTime</i> и <i>endTime</i> .

Табела 1-1: Основне функције OpenMP библиотеке

Променљиве окружења из *OpenMP* библиотеке:

Пример једне променљиве окружења је број нити који ће се користити у току извршавања: ***OMP_NUM_THREADS***. Ова вредност се може поставити из оперативног система или одговарајућим помоћним функцијама.

У наставку је дат једноставан пример Ц програмског кода који користи OpenMP. Покретање програма се обавља на исти начин као и за обичан секвенцијални код.

```
#include <iostream>
#include <omp.h>
using namespace std;

int main()
{
    printf("Starting off in the sequential world.\n");
    #pragma omp parallel
    {
        cout << "Thread number" << omp_get_thread_num() << endl;
    }

    cout << "Back to the sequential world." << endl;
    return 0;
}
```

Паралелни региони:

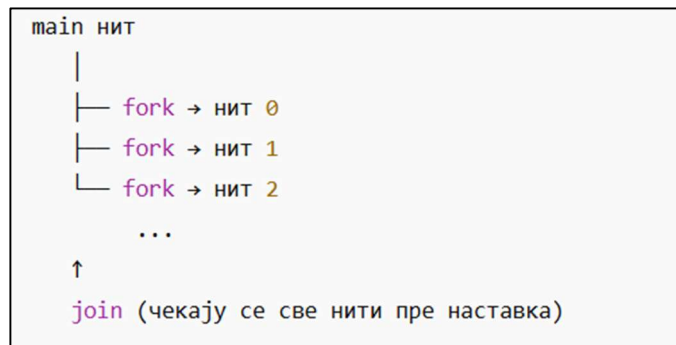
Паралелни регион је основна конструкција у OpenMP-у и представља део програма који се извршава паралелно коришћењем више нити. Сваки OpenMP програм почиње извршавање као секвенцијални програм у једној (главној) нити која започиње извршавање функције *main*. Када наиђе на први паралелни регион (**#pragma omp parallel**), долази до преласка из секвенцијалног у паралелни режим рада.

Приликом уласка у паралелни регион извршавају се следећи кораци:

1. Главна нит (ID = 0) креира скуп нити.
2. Све нити почињу извршавање истог кода унутар паралелног региона.
3. Свака нит има свој јединствени идентификатор (ID).
4. Нити се извршавају независно (редослед није гарантован).
5. На крају региона све нити се синхронизују. Главна нит равноправно учествује у послу и на крају паралелног региона чека на остале нити.
6. Наставља се секвенцијално извршавање (само главна нит)

Овај начин извршавања назива се *fork-join* модел:

- *fork* → креирање више нити,
- *join* → чекање да све нити заврше.



Слика 1: Fork-join модел

✦ Важно: Без фазе „*join*“, програм би могао наставити са радом пре него што су резултати израчунати.

Број нити у програму се може контролисати коришћењем функције `omp_set_num_threads(n)` или коришћењем променљиве окружења `OMP_NUM_THREADS`.

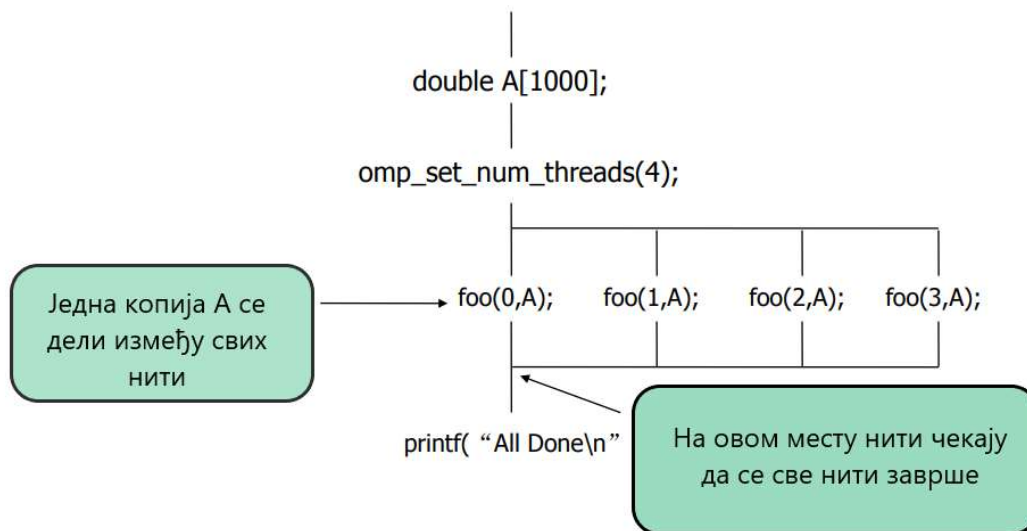
✦ Ако број нити није експлицитно задат, OpenMP користи подразумевану вредност (обично број језгара процесора).

Да би се креирала четири паралелна региона у следећем примеру, свака нит позива функцију *foo* са идентификатором 0 до 3. Све нити извршавају функцију *foo* паралелно. Након што све 4 нити заврше посао, програм наставља са даљим секвенцијалним извршавањем кода, односно исписом „All Done”.

```
double A[1000];
omp_set_num_threads(4);

#pragma omp parallel
{
    int ID = omp_get_thread_num();
    foo(ID, A);
}
```

Паралелни регион се завршава имплицитном баријером што значи да се програм неће наставити док све нити не заврше.



Слика 2 - Паралелни региони

ПИТАЊЕ: Шта би се десило да главна нит не чека да све нити заврше посао?

Паралелни региони представљају најпростију конструкцију у оквиру OpenMP. Недостатак овог приступа је то што корисник мора ручно да имплементира поделу посла, синхронизацију између послова, и слично. Због тога се много чешће користе шаблони вишег нивоа које OpenMP нуди и који ће бити описани у наставку. Иако све нити извршавају исти код, у пракси желимо да поделимо посао тако да свака нит ради различит део. Управо то омогућавају паралелне петље, које ће бити описане у наставку.

Паралелне петље:

OpenMP омогућава једноставну расподелу итерација једне петље на више нити коришћењем директиве за **паралелну петљу**. Да би петља могла да се паралелизује **свака итерација петље мора бити независна од осталих**. OpenMP дели итерације између нити, где свака нит добија део итерација, док редослед извршавања није познат унапред.

```
a[i] = b[i] + c[i];    // добар кандидат за паралелну петљу  
a[i] = a[i-1] + 1;    // не може се користити у паралелној петљи
```

Уколико се петља већ налази у паралелном региону, директива за паралелизацију је:

```
#pragma omp for          // само дели петљу на нити
```

Овом директивом се не прави посебан тим нити спремних за преузимање задатака, већ се подразумева да већ постоје.

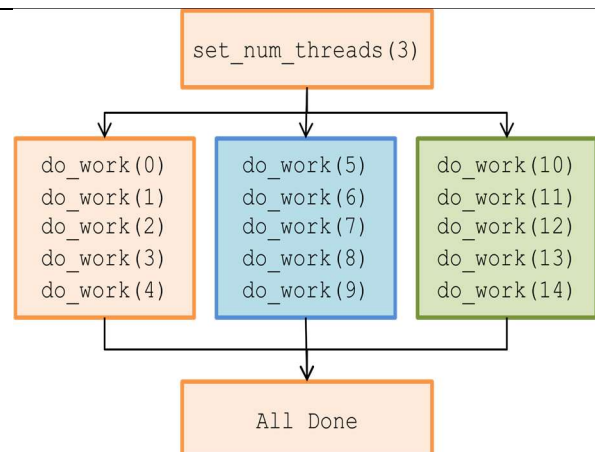
Уколико се налазимо изван паралелног региона, могуће је у оквиру једне директиве направити регион и назначити да читав регион представља петљу. У оба случаја прва инструкција након директиве мора бити *for* петља.

```
#pragma omp parallel for  // креира нити + дели петљу
```

НАПОМЕНА: Уколико има више угњеждених петљи, директива за паралелну петљу се односи само на спољашњу, али је могуће увести паралелизам и у унутрашње петље, додавањем директива за сваку петљу посебно.

У наставку је дат пример једне паралелне петље и графички приказ њеног извршавања. Наранџастом бојом је приказано оно што извршава нит 0, плавом оно што извршава нит 1, а зеленом оно што извршава нит 2.

```
omp_set_num_threads(3);  
  
#pragma omp parallel for  
for (int i = 0; i < 15; i++)  
{  
    do_work(i);  
}  
  
cout << "All Done" << endl;
```



Наредни пример приказује попуњавање вектора коришћењем паралелне петље, као и један од начина **мерења времена** извршавања паралелне петље. Обавезно користити **Release** конфигурацију приликом мерења времена – иначе ће резултати бити непоуздани.

```
const int n = 10;
vector<int> x(n), y(n);

double startTime = omp_get_wtime(); // Dobavi trenutno vreme (sec)

#pragma omp parallel for
for (int i = 0; i < n; ++i)
{
    x[i] = i;
    y[i] = 2 * i;
    do_work(i);
}
double endTime = omp_get_wtime(); // Dobavi vreme nakon petlje

cout << "Time: " << (endTime - startTime) * 1000 << "ms." << endl;
```

НАПОМЕНА: У задацима за вежбу (пример 2, 4) потребно је упоредити време извршавања секвенцијалног и паралелног кода, и израчунати убрзање:

$$UBRZANJE = T_sekvencijalno / T_paralelno$$

Убрзање не мора бити једнако броју нити! Зашто? Због трошкова синхронизације, кеш меморије, контекста и неуједначеног оптерећења.

Иако паралелне петље омогућавају значајно убрзање, оне могу довести до грешака уколико више нити истовремено приступа истим подацима. Овај проблем се назива трка до података (енгл. *Race condition*) и биће објашњен у наставку.

Дељене и приватне променљиве:

У паралелним програмима једно од најважнијих питања је: Да ли све нити користе исту променљиву или свака нит има своју копију? Одговор на ово питање директно утиче на исправност програма.

У оквиру паралелних региона, променљиве могу бити:

- **дељене** (*shared*) – све нити приступају истој променљивој (read/write).
- **приватне** (*private*) – свака нит има своју копију променљиве (којима друге нити немају приступ).

Код дељених променљивих све нити виде исту копију променљиве и могу да читају или пишу у њих. Користе се када је потребно само читање променљиве или ако се реализује упис без конфликта.

За разлику од дељених, код приватних променљивих свака нит има своју копију приватне променљиве, која је невидљива за остале нити (слично прослеђивању параметара функцији по вредности). Из приватне променљиве може да чита или да у њу пише само нит која је њен власник.

```
int x = 0;                                // shared  
  
#pragma omp parallel private(y)  // private
```

OpenMP има следећа подразумевана правила:

- променљиве дефинисане унутар паралелног региона → *private*
- променљиве дефинисане ван региона → *shared*

Клаузуле за прецизирање променљивих:

OpenMP клаузула за прецизирање	Опис
#pragma omp parallel shared (var_list)	Све променљиве из листе су дељене (подразумевано)
#pragma omp parallel private (var_list)	Све променљиве из листе су приватне. Свака нит добија неиницијализовану копију <i>var_list</i>
#pragma omp parallel firstprivate (var_list)	Свака нит добија иницијализовану копију <i>var_list</i> пре уласка у регион
#pragma omp parallel lastprivate (var_list)	Копија из последње итерације се чува и после региона

Уколико желимо да променљиве **x** и **y** буду недељене, иако су декларисане изнад линије у којој се налази директива, потребно је урадити следеће:

```
#pragma omp parallel private(x,y)
```

У наставку следи пример коришћења **private** кључних речи.

```
int main(void){
    int i;
    int x;
    x = 44;
    #pragma omp parallel for private(x)
    for(i = 0; i <= 10; i++){
        x = i;
        printf("Thread num: %d  x: %d\n",omp_get_thread_num(),x);
    }
    printf("x is %d\n", x);
}
```

Након покретања програма, добијамо следеће:

```
Thread num: 0      x: 0
Thread num: 0      x: 1
Thread num: 0      x: 2
Thread num: 3      x: 9
Thread num: 3      x: 10
Thread num: 2      x: 6
Thread num: 2      x: 7
Thread num: 2      x: 8
Thread num: 1      x: 3
Thread num: 1      x: 4
Thread num: 1      x: 5
x is 44
```

Приметићете да променљива **x** има исту вредност на коју је иницијализована пре коришћења паралелне регије. Свака нит има своју копију променљиве **x** и промене унутар петље не утичу на оригиналну вредност, јер је **x** приватна унутар паралелне петље.

Приватне променљиве су **неиницијализоване** на почетку паралелног региона. Ако желимо да омогућимо иницијализацију **x** на вредност која је била додељена пре директиве, можемо користити кључну реч **firstprivate** и тада свака нит добија копију **x = 44**. Такође, ако желимо да ажурирамо променљиве користећи вредност из последње итерације, користимо **lastprivate**.

Претпоставимо да смо желели да задржимо последњу вредност променљиве **x** након паралелне регије. То се може постићи коришћењем **lastprivate (x)** уместо **private(x)**:

```
Thread number: 3      x: 9
Thread number: 3      x: 10
Thread number: 1      x: 3
Thread number: 1      x: 4
Thread number: 1      x: 5
Thread number: 0      x: 0
Thread number: 0      x: 1
Thread number: 0      x: 2
Thread number: 2      x: 6
Thread number: 2      x: 7
Thread number: 2      x: 8
x is 10
```

Добијена вредност за **x** је 10, а не 8. Зашто? То значи да је последња **итерација** задржана, а не последња операција. Шта ће се десити ако заменимо **lastprivate (x)** са **firstprivate (x)**?

```
Thread number: 3      x: 9
Thread number: 3      x: 10
Thread number: 1      x: 3
Thread number: 1      x: 4
Thread number: 1      x: 5
Thread number: 0      x: 0
Thread number: 0      x: 1
Thread number: 0      x: 2
Thread number: 2      x: 6
Thread number: 2      x: 7
Thread number: 2      x: 8
x is 44
```

Многи би у овом случају очекивали вредност 0, тј. да променљива **x** има вредност као при првој итерацији. Коришћење **firstprivate** кључне речи подразумева да свака нит има сопствену инстанцу променљиве која би требало да буде иницијализована на вредност променљиве, јер постоји пре паралелне конструкције. У том случају, свака нит добија своју инстанцу **x** и она је једнака 44.

Следи пример који комбинује све врсте клаузула за приватне и дељене променљиве

```
int factor = 2, n = 10;
int last;

#pragma omp parallel for firstprivate(factor) lastprivate(last)
for (int i = 0; i < n; i++)
{
    int temp = factor * a[i]; // private (локално)
    b[i] = temp;              // shared, али без конфликта
    last = temp;              // lastprivate
}
```

Свака нит добија своју иницијализовану копију променљиве **factor** = 2 јер је дефинисана као `firstprivate` и свака нит ради са својом вредношћу променљиве **factor**. Променљива **temp** је приватна (аутоматски) јер је дефинисана унутар петље и свака нит има своју копију. Променљива **b** је дељена јер је дефинисана ван петље, али свака нит пише у различит **i**. Променљива **last** је се као копија додељује свакој нити, али на крају петље се вредност из последње итерације **i = n-1** копира у `last = factor * a[9]`.

Честа грешка: Коришћење **private** без иницијализације → вредност унутар региона је непозната!

Иако **shared** променљиве омогућавају комуникацију између нити, њихова неконтролисана употреба може довести до грешака.

Одредбе за контролу распоређивања итерација - schedule:

Како би се оптималније реализовала паралелизација, важно је одредити како се итерације распоређују по нитима. OpenMP нуди три начина распоређивања код паралелних петљи:

1. **schedule (static, величина блока)** – Итерације се равномерно и унапред додељују нитима. Први параметар одређује статичку расподелу, док други дефинише величину блокова на које су итерације подељене. Уколико је дефинисана величина блока, количина посла се дели на сегменте те величине. Уколико је на располагању **N** нити, свака нит добија **N**-ти део посла и блокови се статички додељују нитима. Распоређивање се врши редом у круг. Ово је подразумевана опција.

1	7	13	19	25	31
2	8	14	20	26	32
3	9	15	21	27	33
4	10	16	22	28	34
5	11	17	23	29	35
6	12	18	24	30	36

```
#pragma omp parallel for schedule (static, 3)
for(int i = 0; i < 36; i++)
{
    Work(i);
}
```

2. **schedule (dynamic, величина блока)** – Нити саме узимају наредни блок када заврше претходни. Итерације су подељене у блокове чија је величина одређена другим параметром. Читава петља се дели на блокове и они се смештају у ред. Блокови се динамички распоређују на извршавање по нитима, односно чим једна нит заврши посао, преузима први блок из реда. Подразумевана вредност другог параметра (уколико се изостави) је 1.

1	7	13	19	25	31
2	8	14	20	26	32
3	9	15	21	27	33
4	10	16	22	28	34
5	11	17	23	29	35
6	12	18	24	30	36

```
#pragma omp parallel for schedule (dynamic, 1)
for (int i = 0; i < 36; i++)
{
    Work(i);
}
```

3. **schedule (guided, величина блока)** – дефинише се динамичко дељење блокова са опадајућом величином. Иницијално је величина блока већа и смањује се све до величине дефинисане другим параметром. Погодно је за случајеве када су нити међусобно у временском раскораку, где свака итерација захтева отприлике исту количину посла за обраду.

НАПОМЕНА: Коришћењем одредби за распоређивање постижу се боље перформансе, не и исправност петље.

Важно је напоменути да избор распореда итерација (schedule) не утиче на исправност резултата, већ искључиво на перформансе програма. Међутим, чак и ако је распоред итерација оптималан, програм може давати нетачне резултате уколико више нити истовремено приступа истим подацима. Овај проблем је познат као трка до података (енгл. *Race condition*) и представља један од најчешћих проблема у паралелном програмирању.

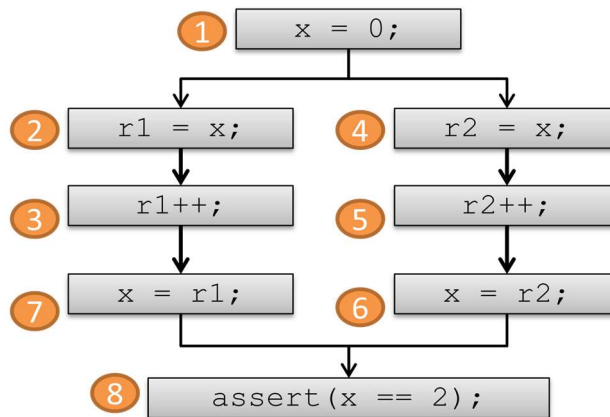
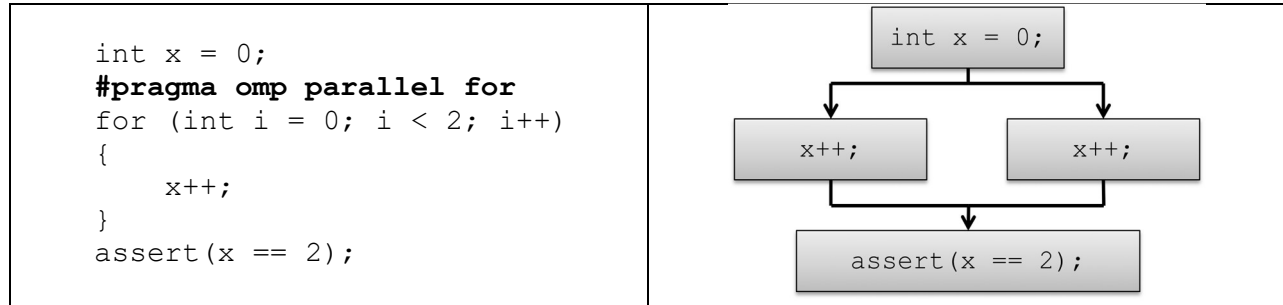
Трка до података:

До сада смо видели како се петље извршавају паралелно и како се итерације распоређују између нити. Међутим, поставља се важно питање:

Шта се дешава ако **више нити** истовремено приступа **истој променљивој**?

Уколико бар једна нит врши упис, може доћи до грешке која се назива **трка до података** (енгл. *Race condition*). У том случају резултат програма постаје непредвидив.

Пример:



Слика 4- Приказ трке до података

На слици 4 приказана је трка до података. Бројеви означавају редослед извршавања инструкција. Овај поредак није увек исти, јер се кораци 2, 3 и 7 извршавају независно од корака 4, 5 и 6. У овом примеру, регистар *r2* преузима вредност променљиве *x* пре него што је она записана (корак 7), те има вредност 0. У кораку 6 и у кораку 7 вредност која је уписана у *x* је број 1, што ће бити и коначан резултат након завршетка петље.

Проблем није у томе што више нити чита исту вредност, већ у томе што више нити истовремено уписује у исту меморијску локацију.

Приликом увођења паралелизма у програмски код неопходно је водити рачуна да се овакве ситуације избегну. Код OpenMP конструкција, потребно је водити рачуна да све итерације паралелне петље, као и све секције унутар једног блока са секцијама буду међусобно независне. За сваку дељену променљиву унутар било ког паралелног региона потребно је обезбедити да јој максимално једна нит приступа зарад писања.

Ово се постиже коришћењем механизма синхронизације, као што су:

- критичне секције (енгл. *Critical*)
- редукција (енгл. *Reduction*)

Критична секција:

Критична секција представља део кода у који у једном тренутку може ући само једна нит. OpenMP нуди неколико начина који обезбеђују да максимално једна нит приступа критичној секцији истовремено. Први начин јесте коришћење **директиве за критичну секцију**:

```
#pragma omp critical (name)
```

Ова директива мора бити задата унутар паралелног региона. Односи се на наредни блок кода (или инструкцију), и означава да **у једном тренутку само једна нит може да извршава дати блок кода**. Уколико нека друга нит из скупа дође до тог блока, чекаће да претходна нит заврши извршавање секције. Директиви се као параметар може проследити назив секције. У том случају, у једном тренутку, само једна нит може да извршава било коју од секција са датим именом.

На следећем примеру дато је како би изгледала заштита критичне секције из претходног примера:

```
int x = 0;
#pragma omp parallel for
for (int i = 0; i < 2; i++)
{
    #pragma omp critical
    x++;
}
assert(x == 2);
```

Недостатак овог решења: програм се успорава, јер нити чекају да претходна нит заврши извршавање секције и губи се смисао паралелизма.

Следећи пример приказује заштиту више критичних секција које су међусобно независне, коришћењем именовања критичних секција. У примеру, паралелни регион као дељене променљиве прима *x* и *y*. Свака нит преузима одговарајући задатак из једног реда, изврши га, па потом преузима задатак из другог реда и изврши и њега. Користећи директиву за критичну секцију обезбеђено је да само једна нит може да ради преузимање задатка из једног реда како се не би десило да две нити преузму исти задатак.

Именовањем секција обезбеђено је да док једна нит преузима задатак из реда *x*, нека друга нит сме да преузима задатак из реда *y* (две независне критичне секције).

```
#pragma omp parallel shared(x, y) private(x_next, y_next)
{
    #pragma omp critical (section_x)
    x_next = pop(x);

    work(x_next);

    #pragma omp critical (section_y)
    y_next = pop(y);

    work(y_next);
}
```

У првом примеру на почетку странице, дат је приказ како уз помоћ директиве за критичну секцију можемо решити проблем када више нити истовремено додаје вредности на дељену променљиву. Међутим ово решење уноси чекање на приступ критичној секцији, кад год се нека друга нит налази у њој. Уколико, као у примеру, критична секција представља читаво тело петље, то ће довести до неефикасног кода. Време извршавања ће бити приближно једнако секвенцијалном коду (у неким случајевима и спорије).

У случајевима као што су сабирање или множење, OpenMP нуди још један концепт који се назива редуковани хиперобјекти или **редуктори**.

Редуктори:

Редуктори омогућавају безбедно комбиновање вредности из више нити и могу се дефинисати за **асоцијативне** операције. Унутар паралелног региона, уместо да све нити приступају једној дељеној променљивој, свака од нити заправо добије копију променљиве (као да је у питању приватна променљива), која је иницијализована на неутрални елемент за дату операцију (0 за сабирање, 1 за множење итд.). Свака нит врши обраду над својом локалном копијом. Током трајања извршавања датог паралелног региона, петље или блока са секцијама, вредност дељене променљиве је недефинисана. У тренутку када се стигне до краја, врши се ажурирање вредности дељене променљиве редуковањем привремених копија. Редуковање заправо представља примену исте операције над привременим резултатима. У случају да је за дату регион, петљу или блок са секцијама задата одредба ***nowait*** редукција се врши на првој наредној баријери.

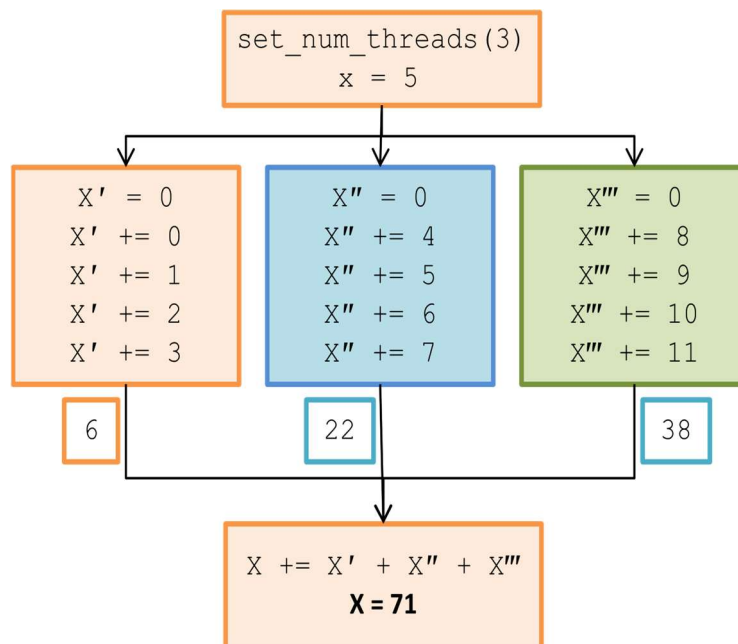
Редуктори се дефинишу на почетку паралелне конструкције додавањем одговарајуће одредбе. Одредба као параметар прима тип операције која ће се вршити над променљивом и листу променљивих.

```
#pragma omp parallel reduction(op : variable-list)
```


Операције које се могу проследити су као параметар *op*: +, *, -, &, ^, |, &&, ||. Над редуктором једног типа могуће је вршити само операцију тог типа.

```
omp_set_num_threads(3);  
  
int x = 5;  
#pragma omp parallel for schedule (static, 4) reduction(+ : x)  
for (int i = 0; i < 12; i++)  
{  
    x += i;  
}  
cout << x;
```

У примеру је направљен скуп од 3 нити, потом је направљена паралелна петља која користи статичко распоређивање са величином бока 4. Променљива *x* је прослеђена као редуктор за сабирање. То значи да ће се петља поделити на 3 блока дужине по 4 итерације, и сваки од блокова ће се извршити на једној од нити. Све 3 нити добиће привремену променљиву *x* иницијализовану на 0.



Слика 1 - Приказ рада редуктора

Свака од нити врши обраду над својом локалном копијом. Након завршетка паралелне петље (на имплицитно уметнутој баријери) врши се редуковање резултата тако што се све привремене копије саберу, и додају на променљиву *x*.

ПИТАЊЕ: Зашто је променљива *x* у паралелној обради добила почетну вредност 0?

Концепт секција:

До сада смо паралелизовали петље тако што смо делили итерације између нити. Међутим, често имамо ситуацију да програм садржи више независних делова који се могу извршавати паралелно. У том случају користимо **секције**.

Секције (енгл. *Sections*) се користе за раздвајање различитих делова кода који се могу паралелно извршавати. У неким ситуацијама независни делови посла могу бити извршавани конкурентно. На пример, мењамо вредности два вектора независно један од другог. У том случају, нит ћемо задужити да обавља обе операције у паралели. Ово се обавља коришћењем секција. Компајлер заказује конкурентно извршавање кода унутар секције. Оваква конструкција омогућава доделу другачије обраде свакој нити.

Примена: Уколико је потребна обрада више различитих вектора, секције су одличан избор.

У примеру који следи направљен је паралелни регион. Унутар региона дефинисан је блок са секцијама коришћењем директиве **#pragma omp sections**.

Након тога, свака од направљених секција представља део кода који се може извршавати у паралели са преосталим секцијама у блоку. Свака секција се распоређује на **једну од нити** из скупа нити за дати паралелни регион.

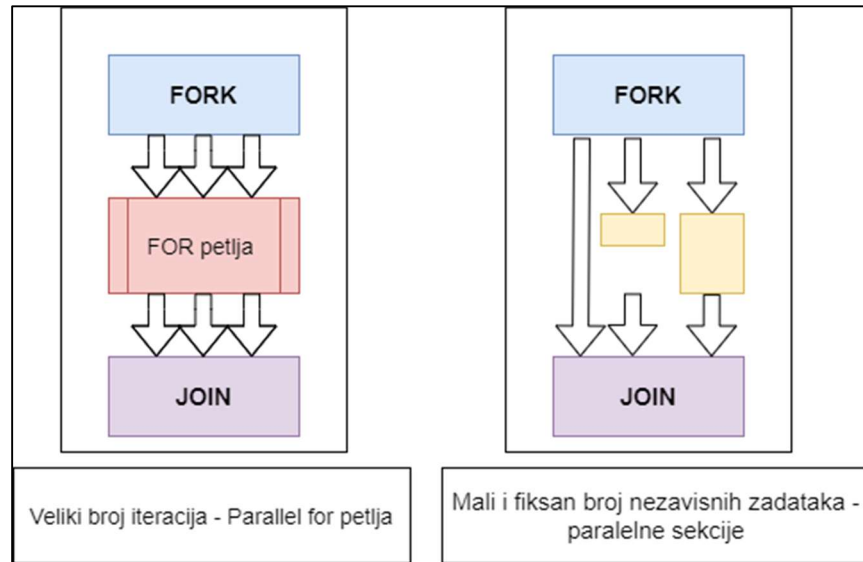
```
#pragma omp parallel
{
    #pragma omp sections
    {
        #pragma omp section
        x_calculation();

        #pragma omp section
        y_calculation();

        #pragma omp section
        z_calculation();
    }
}
```

За директиву за блок секција важе једнака правила за одредбе за дељење променљивих. Такође, као код паралелне петље, могуће је написати блок секција који је уједно и паралелни регион користећи директиву:

```
#pragma omp parallel sections
```



Слика 2 – Паралелна for петља и паралелне секције

Синхронизација нити

Као што је и приказано у претходним примерима, након сваког паралелног региона, паралелне петље или блока са паралелним секцијама – нити се **синхронизују** тј. чека се да све нити из скупа заврше посао пре него што се настави са даљим извршавањем кода. Синхронизација представља механизам којим се координише извршавање више нити са циљем да се обезбеди исправан редослед извршавања. Тачка синхронизације где све нити чекају на завршетак осталих нити назива се **баријера** (енгл. *Barrier*). Баријере је могуће поставити и експлицитно, унутар паралелног региона:

```
#pragma omp barrier
```

Следећи пример приказује начин коришћења баријере. Паралелни регион започиње тако што све нити из скупа позивају функцију *big_calc1* при чему прослеђују свој идентификатор као параметар функције. Потом се чека да све нити заврше обраду и попуне низ A, пре него што иједна нит изврши позив наредне функције. Да је у примеру изостављена баријера, свака нит би извршила позив функције *big_calc2* одмах по завршетку претходне.

```
#pragma omp parallel shared (A, B) private(id)
{
    id=omp_get_thread_num();
    A[id] = big_calc1(id);

    #pragma omp barrier

    B[id] = big_calc2(id, A);
}
```

Баријере које су **имплицитно** (аутоматски) постављене на крај паралелног региона, петље (`parallel for`) или блока са секцијама (`sections`), могуће је уклонити коришћењем одредбе ***nowait***. Уколико ова одредба стоји у директиви, то значи да ће свака нит одмах по завршетку свог посла наставити са даљим извршавањем кода, без чекања на остале.

```
#pragma omp parallel shared (A, B, C) private(id)
{
    #pragma omp for nowait
    for(i = 0; i < N; i++)
    {
        A[i] = big_calc1(i);
    }

    #pragma omp for
    for(i = 0; i < N; i++)
    {
        B[i] = big_calc2(i);
    }

    #pragma omp for
    for(i = 0; i < N; i++)
    {
        C[i] = big_calc3(A, B, i);
    }
}
```

У примеру прва и друга петља су независне. Пошто уз прву директиву стоји одредба ***nowait*** то значи да ће одмах по завршетку свог посла, свака нит започети извршавање наредне петље. Трећа петља за рачунање користи резултат претходне две, те је неопходно сачекати да се претходне две заврше пре него што започне. Изостављањем одредбе ***nowait*** имплицитно се умеће баријера након друге петље.

Када користити **баријеру**?

- када следећи део кода зависи од претходног,
- када сви резултати морају бити спремни,
- код фазног извршавања (pipeline).

Када користити ***nowait***?

- када нема зависности између делова кода,
- када желимо веће перформансе,
- када није потребна синхронизација.

Задаци:

У поставци задатка се налази 6 примера. У сваком примеру је дат секвенцијални код, и исти код који је паралелизован употребом OpenMP директива. Потребно је дате примере отворити, превести и покренути. За сваки од примера потребно је одговорити на следећа питања која следе.

Пре почетка направите нову датотеку са називом ***ImePrezimeRA###_Vežba4.txt***. На почетку датотеке написати име и презиме, након тога у новом реду окружење које сте користили за превођење, оперативни систем који користите, модел процесора на коме се решење извршава и број језгара. Након тога, редом пишете одговоре на питања.

За сваки пример где се тражи мерење времена (убрзања) водити рачуна да користите конфигурацију ***Release*** приликом превођења.

Пример 1: пример паралелног региона и добављања идентификационог броја нити

1. Шта је испис након извршења секвенцијалног кода?
2. Шта је испис након извршења паралелног кода?
3. Колико нити је направљено у паралелном примеру?
4. Напишите измењену функцију *main* тако да направи дупло мање нити од максималног броја дозвољених.

Пример 2: пример паралелне петље. Пример одговара примеру датом на страни 6.

1. Које је време извршавања секвенцијалног кода?
2. Које је време извршавања паралелног кода?
3. Колико је постигнуто убрзање?
4. Да ли је убрзање једнако броју направљених нити? Зашто?

Пример 3: пример коришћења наредбе ***nowait***.

1. Покренути секвенцијални код. Покренути паралелни код више пута. Да ли су излаз из секвенцијалног и паралелног кода исти? Зашто?
2. Шта је потребно урадити да би се грешка исправила?
3. Колико је било убрзање пре исправке? Колико је убрзање након исправке?
4. Шта можете урадити да поправите убрзање након исправке?

Пример 4: пример коришћења редуктора.

1. Да ли је излаз и серијског и паралелног кода исти?
2. Обрисати одредбу *reduction(..)* на линији 22 у паралелној верзији кода. Да ли је резултат исти? Зашто?

3. Изменити код тако да заштитите упис у променљиву користећи директиву за критичну секцију (уместо редуктора). Као одговор на питање (након измена) исписати део кода између позива функција за добављање почетног и крајњег времена извршавања.
4. Колико је убрзање кода када се користи редуктор, а колико када се користи критична секција?

Пример 5: пример коришћења директиве за критичне секције. Илуструје да приступ променљивој није једини узрок критичне секције. Овде проблем настаје због конкурентног уписа у стандардни излаз.

1. Покрените паралелни код више пута, и погледајте излаз. Шта је проблем?
2. Измените код користећи директиве за критичне секције тако да обезбедите да у сваком реду буде исписана само једна реченица. Али тако да и даље нити исписују реченице произвољним редоследом (само је тело петље критична секција, не читава петља). Као одговор на питање напишите код функција *print1*, *print2* и *print3* након измена.

Пример 6: пример рачунања n -тог броја Фибоначијевог низа коришћењем рекурзивног алгорита. Пример илуструје како је могуће раздвојити посао тако да га извршавају две нити у паралели.

1. Покрените серијски и паралелни код. Да ли је резултат исти?
2. Упоредите брзине извршавања, колико убрзање је постигнуто применом паралелизма?
3. Смањите n на 20. Колико је сада убрзање? Зашто?

Додатни задатак (напредни):

1. Изменити пример 6 тако да се посао раздвоји на онолико нити колико је максимално доступно на систему.
Помоћ: можете користити дељену променљиву која представља број доступних нити. У пратећој документацији за OpenMP проверите значење функције *omp_set_nested*.